VULNERABILITY ASSESSMENT AND PENETRATION TESTING REPORT

Compliance Calendar Tracker

Internship Capstone Project

Security Assessment Report

Document Type	VAPT Security Assessment Report
Target Application	Compliance Calendar Tracker
Assessment Type	Black-Box / Grey-Box (Development Stage)
Testing Environment	Localhost / Development
Report Date	May 2026
Classification	Confidential – For Academic Evaluation Only

CONFIDENTIAL

This document contains sensitive security information intended solely for internship evaluation purposes.

Certificate of Completion / Declaration

This is to certify that the Vulnerability Assessment and Penetration Testing (VAPT) documented herein was conducted as part of an internship capstone project. The security assessment was performed on the Compliance Calendar Tracker application — a development-stage project — in a controlled localhost environment.

The assessment was carried out in good faith, adhering to ethical security testing practices and within the boundaries of the defined scope. No production systems, live databases, or external networks were targeted or affected during this assessment.

All findings reported herein are based on genuine testing activities performed using industry-standard tools and the OWASP Testing Methodology. Findings are accurately represented and limitations due to the partial implementation status of the application have been clearly documented.

This report is submitted as part of the internship final evaluation and is intended solely for academic and professional assessment purposes.

Field	Details
Intern Name	***Ravi P***
Institution / University	SKSJTI
Internship Organization	Campuspe
Supervisor / Mentor	Jacob Dennis

1. Executive Summary

This report presents the results of a Vulnerability Assessment and Penetration Testing (VAPT) engagement conducted on the Compliance Calendar Tracker, a web-based application designed to manage and track regulatory compliance activities. The assessment was performed as part of an internship capstone project within a controlled, isolated localhost development environment.

The application under assessment is currently in active development and has not reached production deployment. Accordingly, the scope of testing was limited to currently functional and accessible modules, and findings are presented in the context of a development-stage system. Incomplete modules and features were excluded from the assessment scope.

The assessment methodology was aligned with the OWASP Testing Guide (OTG v4) and encompassed authentication and authorization testing, API security analysis, HTTP security header inspection, cross-site scripting (XSS) evaluation, and error handling review. A combination of automated scanning tools (Burp Suite Community Edition and OWASP ZAP) and manual testing techniques were employed.

1.1 Summary of Findings

A total of six (6) findings were identified during this assessment, distributed across severity levels as follows:

Severity	Count	Findings
Critical	0	*No critical vulnerabilities identified*
High	0	*No high-severity vulnerabilities identified*
Medium	3	Verbose Error Disclosure, Missing CSP Header, Missing Anti-Clickjacking Protection
Low	2	Missing X-Content-Type-Options, Missing Subresource Integrity
Informational	1	JWT Authentication Validation Testing (Token Validation Confirmed Effective)

Notably, the JWT-based authentication mechanism demonstrated robust validation behavior, appropriately rejecting tampered and unauthorized tokens during testing. No evidence of authentication bypass was observed. The findings identified are predominantly security misconfiguration issues commonly encountered in early-stage development environments, where security header configurations and error handling hardening are typically deferred until pre-production stages.

It is recommended that the development team address the medium-severity findings — particularly the missing Content Security Policy and verbose error messaging — prior to any staging or production deployment.

2. Objectives

The primary objectives of this security assessment were defined as follows:

- To systematically identify security vulnerabilities and weaknesses present in the Compliance Calendar Tracker application, within the constraints of its current development stage.
- To evaluate the effectiveness of the implemented authentication and authorization mechanisms, particularly the JWT-based authentication framework.
- To assess the application's adherence to established security best practices as defined by the OWASP Top 10 (2021) framework.
- To review HTTP response headers for the presence of standard browser-enforced security controls.
- To perform controlled cross-site scripting (XSS) payload testing on accessible input fields and parameters.
- To analyze error handling behavior and assess the risk of sensitive information disclosure through verbose error responses.
- To document all identified findings with accurate severity classifications, technical impact analysis, and actionable remediation recommendations.
- To provide a professional, structured security report suitable for academic evaluation and viva presentation.

Note: Given that the application is in an active development phase, the assessment objectives were scoped to reflect what was technically feasible and accessible at the time of testing. Modules and features not yet implemented or accessible were excluded from the assessment scope.

3. Scope of Testing

3.1 In-Scope Assets

Component	URL / Endpoint	Technology
Frontend	http://127.0.0.1:5173	React + Vite
Backend API	http://localhost:8080	Spring Boot (Java)
AI Service	http://localhost:5000 (Internal)	Flask (Python)
Database	localhost:5432 (Internal)	PostgreSQL
Cache / Rate Limiting	localhost:6379 (Internal)	Redis

3.2 Testing Areas Included

- Authentication mechanism testing (JWT token interception, manipulation, and validation)
- Authorization testing (unauthenticated API access, privilege escalation attempts)
- HTTP security header analysis (CSP, X-Frame-Options, X-Content-Type-Options, SRI)
- Cross-site scripting (XSS) testing on accessible input fields
- Error handling and information disclosure review
- Security misconfiguration review (OWASP A05:2021)
- Manual HTTP request manipulation using Burp Suite Repeater

3.3 Testing Limitations and Exclusions

The following constraints were applicable during this assessment:

- The application was assessed in a development-stage environment. Several modules and API endpoints were incomplete or non-functional at the time of testing.
- Testing was limited to the localhost environment; no external network exposure, DNS resolution, or SSL/TLS assessment was performed.
- The Flask-based AI service and Redis cache were tested only indirectly through the primary application interface; direct service exploitation was not in scope.
- Database security assessment (SQL injection depth testing, stored procedure review) was not performed directly.
- No denial-of-service (DoS) or load testing was conducted.
- User enumeration testing was limited due to partial authentication module implementation.
- Business logic vulnerabilities could not be fully assessed due to incomplete module availability.

4. Methodology

The security assessment was conducted using a structured methodology aligned with the OWASP Web Security Testing Guide (WSTG) v4.2 and the OWASP Top 10 (2021) framework. The engagement followed a grey-box testing approach, where limited application context (technology stack and endpoint information) was available to the assessor.

4.1 Testing Phases

Phase 1: Reconnaissance and Information Gathering

Initial reconnaissance was performed to map the application's attack surface. This included identifying available API endpoints, inspecting HTTP request and response headers, examining application behavior across different states (authenticated vs. unauthenticated), and reviewing the technology stack in use. Browser developer tools were used to inspect network traffic, cookies, and local storage artifacts.

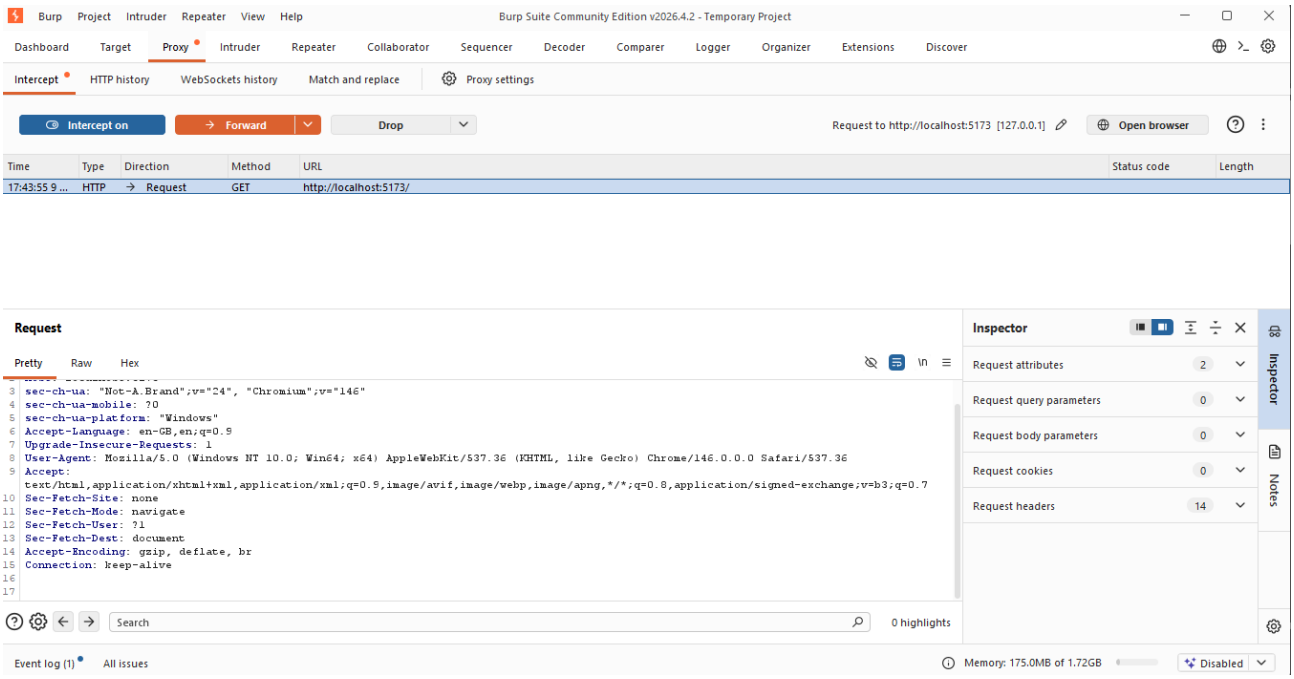
Phase 2: Automated Scanning

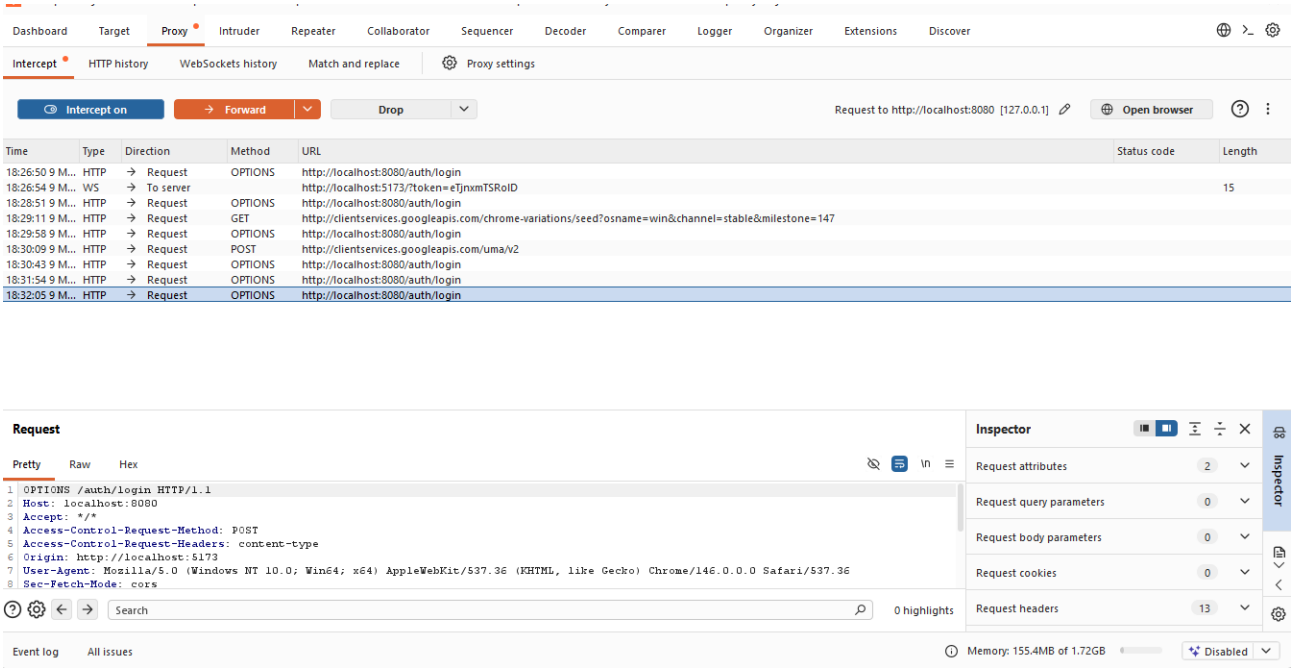
Passive and active scanning was performed using OWASP ZAP to identify common vulnerability patterns. The scanner was configured to analyze HTTP responses for missing security headers, insecure cookie attributes, and potential injection points. Burp Suite Community Edition was used to intercept, inspect, and manipulate HTTP traffic in real time.

Phase 3: Authentication and Authorization Testing

JWT tokens obtained during authenticated sessions were intercepted using Burp Suite Proxy. Multiple manipulation scenarios were tested:

- Forwarding requests with the Authorization header omitted entirely.
- Modifying the JWT payload section (e.g., altering user role claims) without re-signing the token.
- Submitting a structurally valid but cryptographically invalid token.
- Replaying expired tokens to check for token lifetime enforcement.





Phase 4: XSS and Input Validation Testing

Cross-site scripting payloads were manually injected into all accessible input fields and URL parameters. Both reflected and stored XSS scenarios were considered. Standard OWASP XSS test vectors were used, including:

```
<script>alert(1)</script>
```

```
<img src=x onerror=alert(1)>
<svg onload=alert(document.cookie)>
javascript:alert(1)
```

Input fields were observed for direct reflection of injected content. No successful XSS execution was confirmed during the assessment. This may be attributed to React's built-in JSX escaping mechanisms, which by default encode output before rendering in the DOM.

Phase 5: Error Handling and Information Disclosure

Malformed and boundary-case HTTP requests were crafted and submitted to backend API endpoints using Burp Suite Repeater. Response bodies were analyzed for the presence of stack traces, exception class names, internal file paths, database query details, or any other information that could assist an attacker in understanding the application's internal architecture.

Phase 6: Security Header Analysis

All HTTP responses from both the frontend (port 5173) and backend (port 8080) were systematically inspected for the presence and correct configuration of security-relevant response headers. Analysis was supported by OWASP ZAP's passive scan alerts and manual verification using browser developer tools.

4.2 Risk Rating Methodology

Findings are classified using a risk severity matrix derived from the OWASP Risk Rating Methodology, which considers both the likelihood of exploitation and the potential technical and business impact:

Severity	CVSS Score Range	Description
Critical	9.0 – 10.0	Immediate exploitation risk; severe business impact; requires urgent remediation.
High	7.0 – 8.9	Significant exploitation risk with substantial impact; should be prioritized for remediation.
Medium	4.0 – 6.9	Moderate risk with limited impact in isolation; remediation recommended before production.
Low	0.1 – 3.9	Low exploitability or minimal impact; remediation advisable as part of security hardening.
Informational	N/A	Observations or positive security confirmations; no direct vulnerability identified.

5. OWASP Top 10 Mapping

The findings identified during this assessment are mapped to the relevant OWASP Top 10 (2021) categories below. This mapping provides a standardized framework for understanding the vulnerability class and its position within the broader web application security risk landscape.

OWASP Category	Finding	Severity
A05:2021 – Security Misconfiguration	Verbose Error Message Disclosure	Medium
A05:2021 – Security Misconfiguration	Content Security Policy (CSP) Header Not Set	Medium
A05:2021 – Security Misconfiguration	Missing Anti-Clickjacking Protection	Medium
A05:2021 – Security Misconfiguration	X-Content-Type-Options Header Missing	Low
A08:2021 – Software and Data Integrity Failures	Subresource Integrity Attribute Missing	Low
A07:2021 – Identification and Authentication Failures	JWT Authentication Validation Testing	Informational

Note: No findings mapping to A01 (Broken Access Control), A02 (Cryptographic Failures), A03 (Injection), A04 (Insecure Design), A06 (Vulnerable Components), A09 (Security Logging Failures), or A10 (Server-Side Request Forgery) were confirmed during the assessment scope. This is partly attributable to the limited testing surface available in the development-stage environment.

6. Tools Used

Tool	Version / Edition	Purpose
Burp Suite Community Edition	2024.x	HTTP proxy, request interception, repeater-based manual testing, JWT token manipulation
OWASP ZAP (Zed Attack Proxy)	2.15.x	Automated passive and active scanning, security header analysis, vulnerability identification
Browser Developer Tools	Chrome / Firefox	Network traffic inspection, cookie analysis, local storage review, DOM inspection
Manual OWASP Testing Methodology	WSTG v4.2	Structured test case execution aligned with OWASP Web Security Testing Guide
curl / Postman (auxiliary)	—	Supplementary API request crafting and response inspection

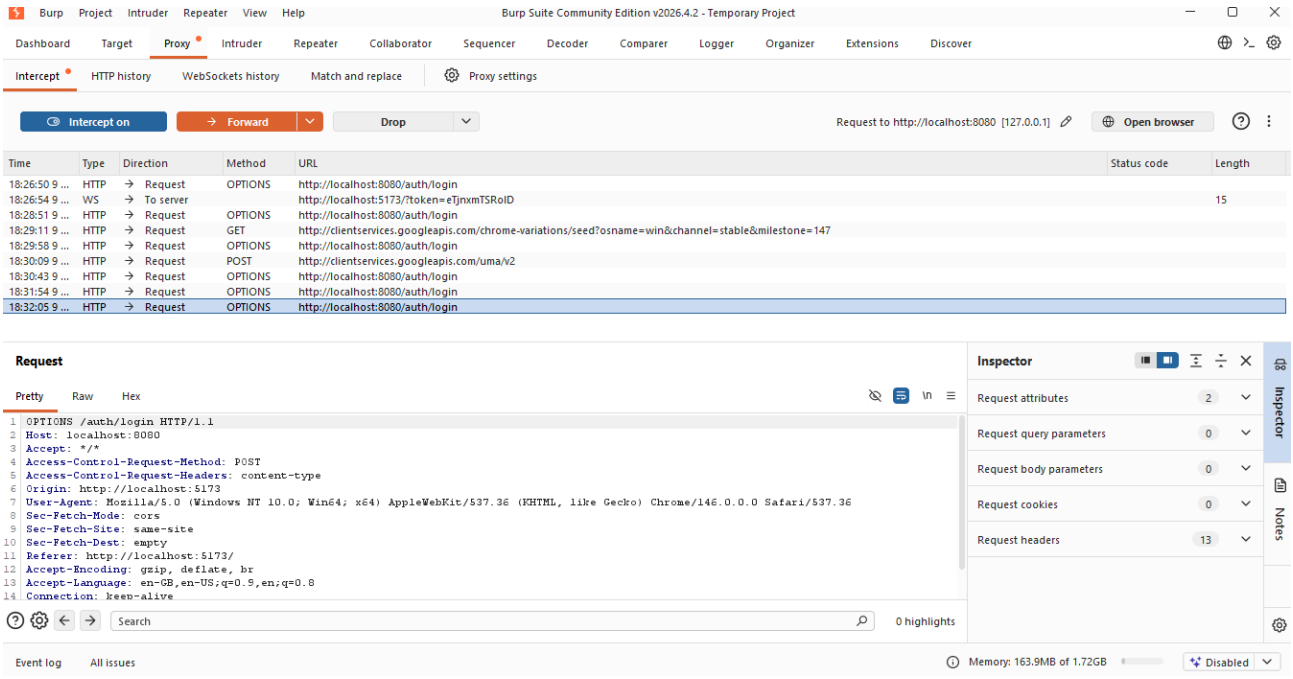
7. Detailed Findings

This section presents the detailed technical findings identified during the assessment. Each finding includes a description, potential impact, steps to reproduce (where applicable), evidence observations, and specific remediation recommendations.

Finding 1: Verbose Error Message Disclosure	
Severity	**Medium**
OWASP Category	A05:2021 – Security Misconfiguration
CVSS Score (Approx.)	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Description	When API requests were crafted with malformed parameters or manipulated payloads, the Spring Boot backend returned verbose HTTP 500 error responses. These responses included exception class names, method signatures, and in some cases internal file paths, indicating that the application's global exception handling is not configured to suppress detailed error information.
Impact	Verbose error messages provide an attacker with valuable reconnaissance information. Exposed class names and stack traces can reveal the application's framework version, internal package structure, and implementation logic — information that can be leveraged to craft more targeted attacks.
Steps to Reproduce	1. Intercept an authenticated API request using Burp Suite Proxy. 2. Forward the request to Burp Repeater. 3. Modify the request body to include malformed JSON or omit required fields. 4. Send the request and observe the HTTP 500 response body for stack trace details.

Evidence / Observation

During testing, an HTTP POST request to a backend API endpoint was modified to include an invalid payload structure. The resulting HTTP 500 response contained a Java exception trace, exposing the exception class (e.g., org.springframework.http.converter.HttpMessageNotReadableException) and internal method call hierarchy. This behavior was consistently reproducible across multiple endpoints during the assessment.



Recommendation

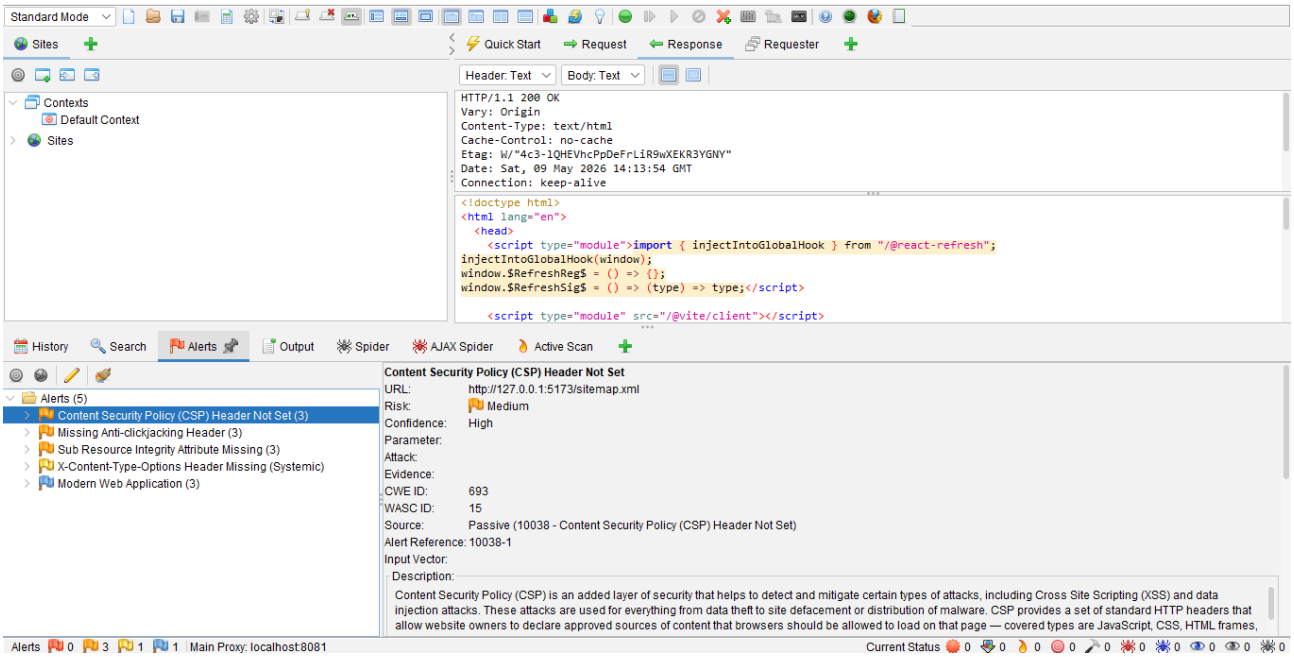
Implement a centralized global exception handler using Spring Boot's @ControllerAdvice/@RestControllerAdvice pattern. The handler should catch all unhandled exceptions and return a standardized, non-descriptive error response to the client (e.g., { "error": "An unexpected error occurred. Please try again later." }). Full exception details should be logged server-side for debugging purposes but never exposed in API responses. This is a standard pre-production hardening measure.

```
@RestControllerAdvice public class GlobalExceptionHandler {
@ExceptionHandler(Exception.class)     public ResponseEntity<ErrorResponse>
handleAll(Exception ex) {         log.error("Unhandled exception", ex);
return ResponseEntity.status(500).body(new ErrorResponse("Internal server error"));
} }
```

Finding 2: Content Security Policy (CSP) Header Not Set	
Severity	**Medium**
OWASP Category	A05:2021 – Security Misconfiguration
CVSS Score (Approx.)	5.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
Description	HTTP responses from both the frontend (port 5173) and the backend API (port 8080) did not include a Content-Security-Policy (CSP) response header. CSP is a browser-enforced security mechanism that restricts the sources from which the browser is permitted to load resources such as scripts, styles, images, and frames.
Impact	The absence of a CSP header significantly increases the application's exposure to client-side injection attacks, including Cross-Site Scripting (XSS). Without CSP, a successful XSS attack would not be mitigated by any browser-level policy, allowing injected scripts to execute without restriction.
Steps to Reproduce	1. Navigate to the application at http://127.0.0.1:5173. 2. Open browser Developer Tools (F12) > Network tab. 3. Inspect any HTTP response headers for the presence of 'Content-Security-Policy'. 4. Confirm the header is absent. Alternatively, OWASP ZAP passive scan reports this finding automatically.

Evidence / Observation

OWASP ZAP passive scan consistently flagged the absence of the Content-Security-Policy header across all inspected responses during the assessment. Manual verification using browser Developer Tools confirmed the finding. No CSP header was present in any application response headers.



Recommendation

Implement a Content Security Policy header appropriate for the application's resource requirements. For a React-based frontend served via Vite, a starting point would be:

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self' http://localhost:8080; frame-ancestors 'none';
```

For the Spring Boot backend, CSP headers can be configured via Spring Security's header configuration. The policy should be refined progressively, using CSP reporting mode (Content-Security-Policy-Report-Only) initially to identify policy violations without breaking functionality.

Finding 3: Missing Anti-Clickjacking Protection	
Severity	**Medium**
OWASP Category	A05:2021 – Security Misconfiguration
CVSS Score (Approx.)	4.7 (AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:N/A:N)
Description	The application does not implement anti-clickjacking protections. Neither the X-Frame-Options response header nor the frame-ancestors directive within a Content Security Policy was present in any HTTP response. This means the application's pages can be embedded within an <iframe> on an attacker-controlled website.
Impact	An attacker could embed the application within a transparent or opaque iframe on a malicious website, tricking authenticated users into performing unintended actions (clickjacking). In a compliance tracking context, this could result in unauthorized modification of compliance records or approval workflows.
Steps to Reproduce	1. Inspect HTTP response headers from any application page for X-Frame-Options. 2. Confirm the header is absent. 3. Alternatively, create a simple HTML file with: <iframe src="http://127.0.0.1:5173"></iframe> and open it locally to confirm the page renders inside the iframe.

Evidence / Observation

OWASP ZAP passive scan reported the absence of anti-clickjacking protection headers. Manual verification confirmed that the application could be successfully embedded within an iframe in a local test HTML file, with the full application interface rendered inside the frame. No browser-enforced frame restriction was observed.

Recommendation

Implement one of the following controls. The preferred approach is to use the frame-ancestors CSP directive (which supersedes X-Frame-Options in modern browsers):

```
Content-Security-Policy: frame-ancestors 'none';
```

Alternatively, or additionally for legacy browser compatibility:

```
X-Frame-Options: DENY
```

In Spring Boot, this can be configured via Spring Security:

```
http.headers().frameOptions().deny();
```

Finding 4: X-Content-Type-Options Header Missing	
Severity	**Low**
OWASP Category	A05:2021 – Security Misconfiguration
CVSS Score (Approx.)	3.1 (AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N)
Description	The application's HTTP responses did not include the X-Content-Type-Options security header. This header instructs browsers not to perform MIME-type sniffing — a process by which browsers attempt to infer the content type of a response when it is ambiguous or missing.
Impact	In the absence of this header, certain browsers may misinterpret the content type of a response and execute content as a different type than intended. For example, a response that serves JSON or plain text could potentially be interpreted as executable JavaScript in certain contexts, facilitating MIME confusion attacks.
Steps to Reproduce	1. Use Burp Suite or browser Developer Tools to inspect any HTTP response from the application. 2. Verify the absence of the X-Content-Type-Options header in the response headers.

Evidence / Observation

OWASP ZAP passive scan identified the absence of the X-Content-Type-Options header across all inspected HTTP responses. This was corroborated by manual inspection of response headers using the browser Developer Tools Network panel.

Recommendation

Add the following HTTP response header to all application responses:

```
X-Content-Type-Options: nosniff
```

In Spring Boot with Spring Security, this header is enabled by default. Verify that it is not being suppressed:

```
http.headers().contentTypeOptions();
```

Finding 5: Subresource Integrity (SRI) Attribute Missing	
Severity	**Low**
OWASP Category	A08:2021 – Software and Data Integrity Failures
CVSS Score (Approx.)	3.7 (AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N)
Description	External JavaScript libraries and stylesheets referenced in the application's HTML did not include Subresource Integrity (SRI) attributes. SRI is a browser security feature that allows the browser to verify that fetched resources have not been tampered with by comparing a cryptographic hash of the fetched resource against a hash provided in the HTML tag.
Impact	If an external CDN or third-party resource host is compromised, a supply chain attack could serve modified malicious scripts to application users. Without SRI, the browser has no mechanism to detect or reject such tampering.
Steps to Reproduce	1. Inspect the application's HTML source (view-source: or DevTools > Elements). 2. Identify any <script> or <link> elements referencing external URLs (CDN-hosted resources). 3. Verify the absence of integrity and crossorigin attributes on these elements.

Evidence / Observation

OWASP ZAP passive scan reported the absence of integrity attributes on externally referenced resources. Inspection of the application's HTML source confirmed that at least one external resource reference lacked the integrity and crossorigin attributes required for SRI enforcement.

[Insert Screenshot – OWASP ZAP Alert: 'Subresource Integrity Attribute Missing']

Recommendation

For any externally hosted scripts or stylesheets, generate and include SRI hashes. Example:

```
<script src="https://cdn.example.com/library.js"   integrity="sha384-<base64-encoded-hash>"   crossorigin="anonymous"></script>
```

SRI hash values can be generated using the SRI Hash Generator at https://www.srihash.org or via OpenSSL:

```
openssl dgst -sha384 -binary library.js | openssl base64 -A
```

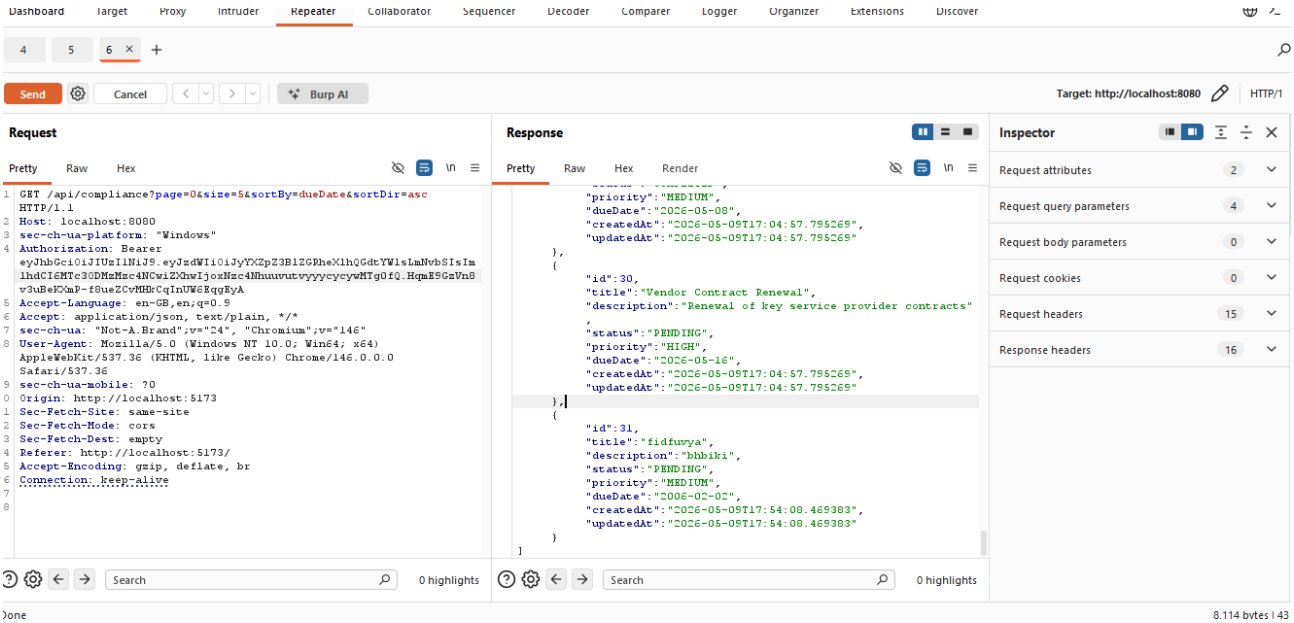
Note: If the application is transitioning to a fully bundled build (e.g., via Vite's production build), externally referenced CDN resources may be eliminated entirely, which would natively resolve this finding.

Finding 6: JWT Authentication Validation Testing (Informational)	
Severity	**Informational**
OWASP Category	A07:2021 – Identification and Authentication Failures
CVSS Score (Approx.)	N/A – No vulnerability confirmed
Description	JWT-based authentication was tested through a series of token manipulation scenarios using Burp Suite. This finding documents the results of those tests, which confirmed that the backend authentication mechanism is functioning correctly. Testing was performed to validate the presence and effectiveness of server-side JWT validation.
Observation	All tested JWT manipulation scenarios were handled correctly by the backend. Requests with missing, modified, or structurally invalid tokens received appropriate HTTP 401 (Unauthorized) responses. No authentication bypass was achieved during the assessment.

Evidence / Observation

The following JWT manipulation scenarios were tested using Burp Suite Repeater:

- Request with Authorization header completely removed: HTTP 401 Unauthorized returned.
- Request with a JWT token with a modified payload section (user ID and role claims altered without re-signing): HTTP 401 Unauthorized returned, confirming signature validation is enforced.
- Request with a syntactically malformed JWT (random Base64 string): HTTP 401 Unauthorized returned.
- Request with a structurally valid token using the 'alg: none' technique (where supported by libraries): HTTP 401 Unauthorized returned.



The backend Spring Boot application correctly validated the cryptographic signature of incoming JWT tokens and rejected any token that did not conform to expected signature parameters. This indicates that the JWT

secret key is not trivially guessable and that the application is not vulnerable to the 'algorithm confusion' (alg:none) attack.

Recommendation

The current authentication implementation demonstrates appropriate security controls. The following recommendations are offered as best-practice guidance for continued hardening:

- Ensure JWT tokens are configured with a reasonable expiration time (exp claim), typically 15–60 minutes for access tokens.
- Implement token refresh mechanisms with longer-lived refresh tokens stored securely.
- Maintain and validate the iss (issuer) and aud (audience) claims within the JWT to prevent token reuse across services.
- Ensure the JWT secret key or RSA private key is rotated periodically and is not embedded in source code or configuration files committed to version control.
- Consider implementing a token revocation mechanism (e.g., a token blacklist via Redis) for logout and session invalidation scenarios.

8. Risk Rating Summary

The table below provides a consolidated risk rating summary for all findings identified during this assessment.

#	Finding Title	Severity	OWASP Mapping	Remediation Priority
F1	Verbose Error Message Disclosure	**Medium**	A05:2021	High – Address before staging deployment
F2	Content Security Policy (CSP) Header Not Set	**Medium**	A05:2021	High – Implement before production
F3	Missing Anti-Clickjacking Protection	**Medium**	A05:2021	High – Implement before production
F4	X-Content-Type-Options Header Missing	**Low**	A05:2021	Medium – Part of security hardening
F5	Subresource Integrity Attribute Missing	**Low**	A08:2021	Medium – Implement for external resources
F6	JWT Authentication Validation Testing	**Informational**	A07:2021	No action required – Continue monitoring

9. Consolidated Recommendations

The following recommendations are presented in priority order based on their associated risk severity and potential impact on the application's security posture.

9.1 High Priority – Address Before Staging Deployment

R1: Implement Centralized Exception Handling (Related to F1)

- Configure a @RestControllerAdvice global exception handler in Spring Boot.
- Return standardized, non-descriptive error messages to API clients (e.g., HTTP 500 with a generic error body).
- Log full exception details server-side using a structured logging framework (e.g., SLF4J + Logback).
- Ensure development-mode error handling is disabled or stripped in staging and production builds.

R2: Implement Content Security Policy Headers (Related to F2)

- Configure CSP headers on both the frontend (Vite server / Nginx) and the Spring Boot backend.
- Begin with a report-only policy to identify violations without impacting users.
- Progressively tighten the policy, eliminating 'unsafe-inline' and 'unsafe-eval' directives where feasible.
- Reference the OWASP CSP Cheat Sheet for guidance on policy construction.

R3: Implement Anti-Clickjacking Protection (Related to F3)

- Add X-Frame-Options: DENY to all HTTP responses.
- Include the frame-ancestors 'none' directive within the CSP header.
- Verify implementation using browser DevTools or a third-party security header checker.

9.2 Medium Priority – Address During Security Hardening

R4: Set X-Content-Type-Options Header (Related to F4)

- Verify that Spring Security's default content type options header is enabled and not being suppressed.
- Add X-Content-Type-Options: nosniff to all API responses.

R5: Implement Subresource Integrity (Related to F5)

- Audit all HTML templates and index files for external CDN resource references.
- Generate SHA-384 SRI hashes for each external resource and add integrity and crossorigin attributes.
- Consider bundling critical dependencies locally via Vite to eliminate external CDN dependencies entirely.

9.3 Ongoing Security Best Practices

R6: JWT Security Hardening (Related to F6)

- Review and enforce appropriate token expiry times (access tokens: 15-60 minutes; refresh tokens: 7-30 days).
- Validate iss, aud, and exp claims on every token validation.
- Store JWT signing secrets in a secrets management solution (e.g., HashiCorp Vault, AWS Secrets Manager) rather than application configuration files.
- Implement a token revocation/blocklist mechanism using the existing Redis infrastructure.

R7: Implement HTTPS Before Any Deployment Beyond Localhost

- Configure TLS/SSL certificates before any staging or production deployment.
- Enforce HTTPS using HTTP Strict Transport Security (HSTS) headers.
- Set the Secure flag on all session cookies.

R8: Establish a Dependency Vulnerability Management Process

- Integrate dependency scanning tools (e.g., OWASP Dependency-Check, Snyk, npm audit) into the build pipeline.
- Regularly review and update third-party library versions to address known CVEs.

10. Conclusion

This Vulnerability Assessment and Penetration Testing engagement was conducted on the Compliance Calendar Tracker application — a development-stage web application comprising a React/Vite frontend, a Spring Boot backend API, a Flask-based AI service, and a PostgreSQL database supported by Redis caching.

The assessment was performed within the constraints of a localhost development environment, with testing scope limited to currently functional and accessible modules. A total of six (6) findings were identified and documented:

- Three (3) findings of Medium severity, all relating to missing HTTP security headers and verbose error handling — both categories being common in development-phase applications where security hardening is typically deferred.
- Two (2) findings of Low severity, involving missing browser-level security controls that reduce exposure to MIME sniffing and supply chain attacks.
- One (1) Informational finding, confirming that the JWT-based authentication mechanism effectively validates tokens and rejects unauthorized or tampered requests.

Importantly, no critical or high-severity vulnerabilities were identified during this assessment. The absence of such findings is consistent with the application's current development stage and the limited attack surface exposed in a localhost environment. The authentication layer, in particular, demonstrated a sound implementation that correctly enforced cryptographic token validation across all tested scenarios.

The findings identified are realistic and expected for a development-phase application. They represent configuration gaps rather than fundamental architectural weaknesses, and the majority can be addressed through relatively straightforward remediation steps — most of which involve the configuration of standard HTTP response headers and exception handling patterns.

It is recommended that the identified medium-severity findings — verbose error disclosure, missing Content Security Policy, and missing anti-clickjacking headers — be prioritized for remediation prior to any deployment beyond the localhost development environment. Addressing these issues proactively will significantly improve the application's security posture for staging and eventual production deployment.

11. Future Security Improvements

The following recommendations outline a strategic roadmap for enhancing the application's security posture as it progresses from the current development stage toward staging and production environments. These recommendations extend beyond the findings identified in this assessment and represent proactive security investments.

11.1 Pre-Staging Security Milestones

- Complete implementation of HTTP security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy) across all application components.
- Conduct a formal security configuration review of Spring Boot Security settings, including CORS policy, CSRF protection status, and session management.
- Implement comprehensive server-side input validation and sanitization across all API endpoints to mitigate injection vulnerabilities.
- Enable and configure Spring Boot's security audit logging to capture authentication events, access control decisions, and administrative actions.

11.2 Authentication and Session Management Enhancements

- Implement multi-factor authentication (MFA) for privileged user roles.
- Introduce account lockout mechanisms to mitigate brute-force attacks on authentication endpoints.
- Establish a formal session management policy including idle timeout enforcement and secure logout with token invalidation.
- Migrate JWT secret management to a dedicated secrets management solution.

11.3 API Security Hardening

- Implement API rate limiting at the gateway or application level, leveraging the existing Redis infrastructure.
- Define and enforce strict API input schemas with request body validation (e.g., using Spring Validation annotations with @Valid).
- Audit all API endpoints for proper authorization checks, ensuring that object-level and function-level access controls are enforced consistently.
- Consider implementing API versioning and deprecation policies.

11.4 Comprehensive Security Testing

- Conduct a full-scope VAPT assessment in a staging environment once all application modules are complete and accessible.
- Integrate automated security testing (DAST) into the CI/CD pipeline using tools such as OWASP ZAP or Nuclei.
- Perform source code security analysis (SAST) using tools such as SonarQube or Semgrep.
- Conduct dedicated business logic testing once all application workflows are implemented.
- Consider engaging a third-party security firm for an independent penetration test prior to production launch.

11.5 Infrastructure and Deployment Security

- Configure TLS 1.2+ with strong cipher suites for all network communication before any external deployment.
- Implement network segmentation between application components (frontend, backend, database, cache).
- Configure database access controls to enforce the principle of least privilege for the application's database user.
- Establish a security patch management process covering all application dependencies and infrastructure components.
- Implement runtime application self-protection (RASP) or a Web Application Firewall (WAF) for production deployment.

11.6 Security Awareness and Process

- Adopt a Secure Software Development Lifecycle (SSDLC) framework, integrating security review checkpoints into the development workflow.
- Conduct threat modeling exercises as new features and modules are designed.
- Establish a vulnerability disclosure and incident response process before production launch.

Implementation of these recommendations will position the Compliance Calendar Tracker to meet industry security standards appropriate for a production application handling compliance-sensitive organizational data.